

Isaac Levinsky

Rust Systems Engineer | Cross-Platform Rust Cores, Self-Hosted AI Infrastructure, Shipped Products

ilevinsky36@gmail.com | github.com/IsaacLevinsky | mcmlv1.com | Tampa Bay Area, FL | Remote, async-preferred | Full-time or contract

Rust-first systems engineer who writes product logic once as a shared Rust core and ships it behind native shells on **Android, macOS, Windows, and Linux**. Strongest in 0-to-1 work: designing under real constraints, prototyping fast, and validating through tests, benchmarks, and real-world behavior. Runs all AI on owned hardware - customer-facing serving, private inference, and LLM training from scratch in Rust - with no cloud spend. I use AI heavily for implementation; I own design, integration, and verification.

EXPERIENCE

Founder & Software Engineer - MCMLV1 LLC (mcmlv1.com)

Jan 2026 - Present

- Shipped **15 Android apps on Google Play**, 4 on Amazon Appstore, **Talk Together** (offline AI translation) on Samsung Galaxy Store, and **QR Code Link Me** on the Microsoft Store (macOS version in App Store review) - offline-first, no ads or tracking.
- Private Dictation: Offline AI** (on-device Whisper): **3.27K device acquisitions, 936 active installs** in 6 months.
QR Code Generator Offline: 1.28K acquisitions, 381 active.
- MerlinChat**, the live sales and support chatbot on mcmlv1.com (try it): **HAProxy load-balances concurrent users across multiple GPU workstations** running local LLMs - no third-party inference API; also in my Android apps.

Independent Software Engineer - self-directed R&D

2021 - 2025

- Built the Rust, CUDA, Android, and ML foundations; held all releases until forming the LLC, for liability protection.

RUST SYSTEMS

Cross-platform Rust cores - one engine, native shells on four platforms

- Core engines written once in Rust and embedded through FFI bindings in each platform's native shell: Kotlin and C#/.NET on Android, **Swift on macOS**, and native desktop apps on Windows and Linux.
- One tested codebase owns behavior and performance, so fixes land everywhere at once. Example: **QR Code Link Me** - one Rust core behind Windows and macOS shells; DocEngine powers MCMLV1 Viewer.

CodeLoop - autonomous, air-gapped coding platform (Rust core; Swift and Tauri frontends)

- Builds complete software programs autonomously and fully offline, with built-in data-science analysis and visualization. One Rust core behind a native Swift GUI on macOS and a Tauri frontend on Windows and Linux.
- Inference: MLX on a **32GB M5 MacBook Pro** with models I quantized myself, llama.cpp on Windows/Linux, and air-gapped LAN endpoints that offload larger models to two GPU workstations.

RustMind - LLM training stack designed from scratch in Rust (Burn, a Rust deep-learning framework)

- Built the complete training path from first principles: model architecture, byte-level BPE tokenizer, corpus pipeline, validation design, and GPU training - no PyTorch anywhere in the training loop.
- Parameter ladder from 21M to 125M: proved generative training at 125M and a 72M encoder-decoder; trained the **21M encoder to production quality**, where it runs in shipped software today.
- Root-caused a generation-collapse failure to non-stratified validation splits and over-memorized templated data; rebuilt the corpus pipeline around document-aware chunking with explicit train/validation separation.
- Rust generics over CUDA/CPU backends: one codebase from GPU training to CPU inference. Public proof: **rustmind-mini-embed-base** (MIT), tokenizer to INT8 ONNX semantic search, reproducible on CPU.

Private inference assistant - end-to-end private LLM chat (Rust + Kotlin)

- Rust gateway on an owned GPU workstation serving **gpt-oss-20b at ~72 tokens/sec**; Android client is a Kotlin/Jetpack Compose shell over a Rust core handling streaming and on-device history.
- WireGuard-only transport with no public endpoint and inference bound to localhost; retrieval over a SQLite FTS5/BM25 knowledge cache, with prompts ordered for llama.cpp prefix-cache reuse on long conversations.

LookingGlass - zero-dependency Rust workspace engine

- Compiled-only repo analysis: project classification, language detection, dependency and reference graphing, automated audit reports. **3,441 files in ~47ms**, single binary. Powers RustMind's corpus export with zero false positives across 30+ repositories.

PRODUCTION AI & GPU SYSTEMS

Local inference & GPU engineering

- Bosun**: model routing across 8B-120B tiers via llama-swap; on-device Android LLMs (LiteRT-LM) at sub-2s response.
- Hardware-aware deployment**: INT8/Q4 quantization; diagnosed and fixed FP16/BF16 training instability on Pascal GPUs.
- Orion (C++/CUDA)**: GPU data cleaning and dedup at **~901K rows/sec** (1.15M rows in 1.276s), CPU/GPU cross-validated.

TECHNICAL SKILLS

Languages: Rust | Swift | C++/CUDA | C#/.NET | Kotlin | Python | TypeScript/JavaScript | Go | SQL | Bash

Platforms: Android (Jetpack Compose, .NET) | macOS / Apple silicon (Swift) | Windows | Linux | Tauri | Rust FFI / UniFFI

AI & inference: llama.cpp/llama-swap | MLX | gpt-oss | Burn | PyTorch | Hugging Face | ONNX | LiteRT-LM | ML Kit | Whisper

Infrastructure: HAProxy | WireGuard | Cloudflare (Pages / Tunnel / R2) | Linux / systemd | Docker | SQLite (WAL / FTS5) | Stripe | Git

APPROACH

Give me a hard objective and constraints; I design, build, measure, and iterate until it works. Best with end-to-end ownership in remote, async, output-measured roles.